

Insurance Claim RAG System Architecture

Executive Summary

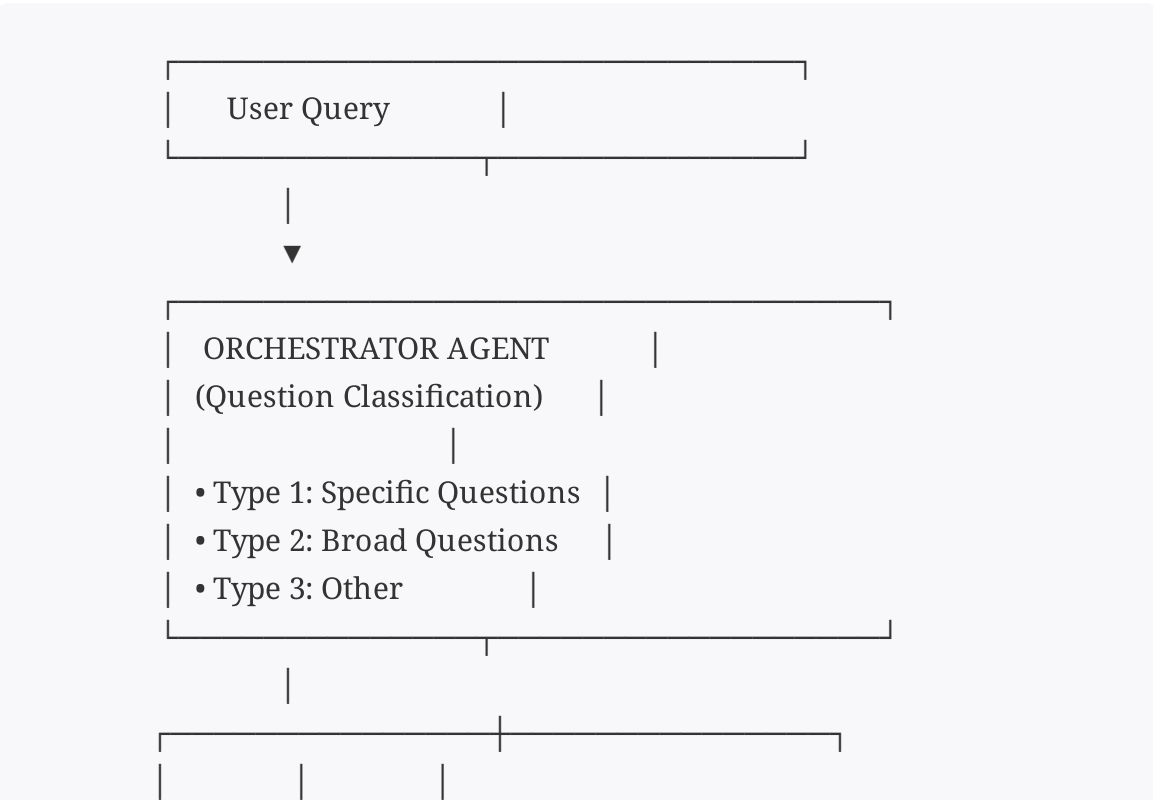
The Insurance Claim RAG (Retrieval-Augmented Generation) System is a sophisticated multi-agent architecture built with LangChain that intelligently processes and queries insurance claim documents. This document describes the system's architecture, components, and workflow.

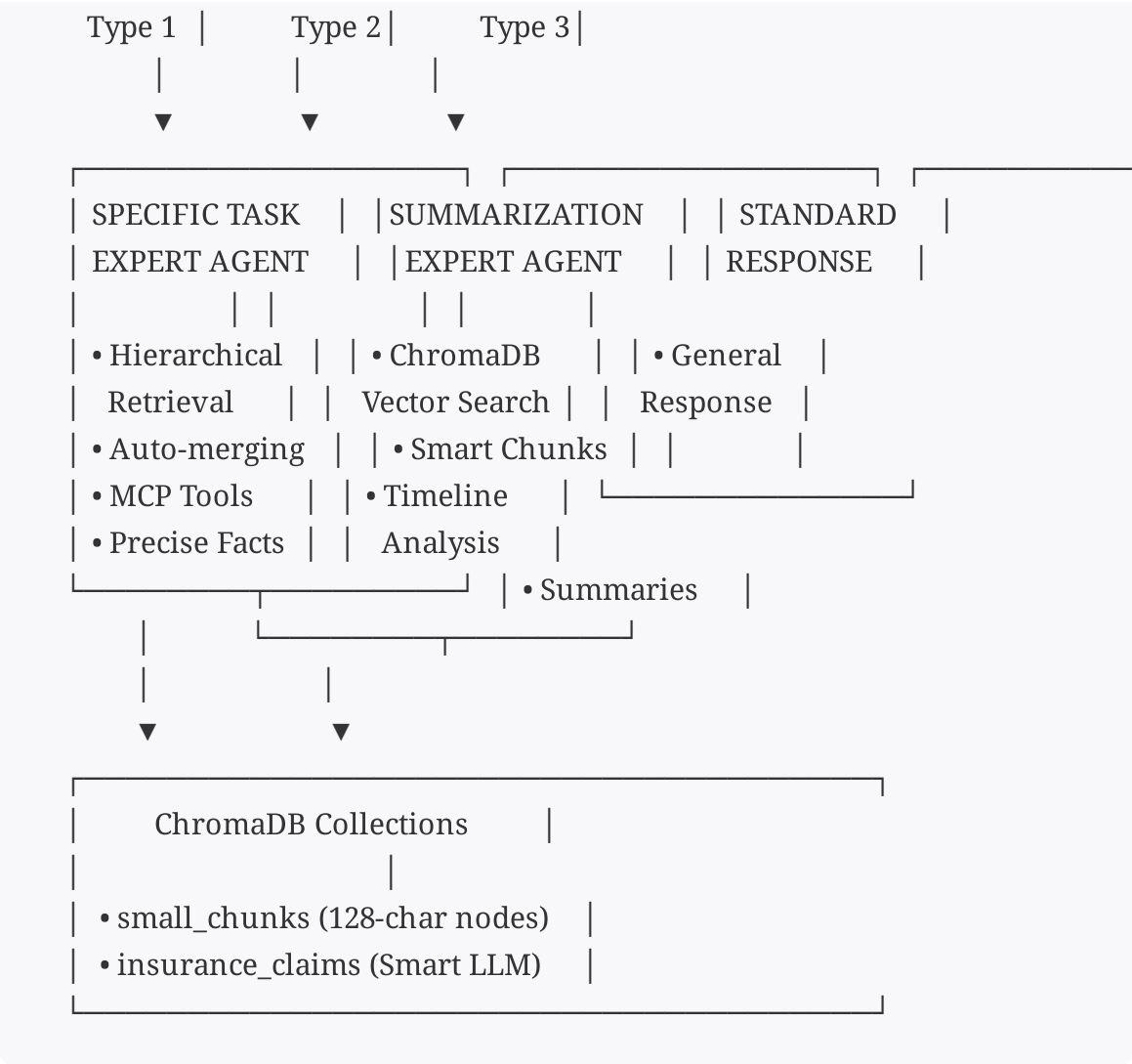
1. System Overview

The system implements a **two-agent architecture** with intelligent question classification and specialized routing:

- **Orchestrator Agent:** Classifies incoming questions and routes them to appropriate specialists
- **Summarization Expert Agent:** Handles broad, complex questions requiring comprehensive analysis
- **ChromaDB Vector Database:** Stores intelligent document chunks for semantic search
- **LlamaIndex Framework:** Provides hierarchical retrieval with auto-merging capabilities

2. Architecture Diagram





3. Component Details

3.1 Orchestrator Agent

Purpose: Central routing and classification component

Functionality:

- Analyzes incoming user questions using GPT-3.5-turbo
- Classifies questions into three categories
- Routes to appropriate specialist agents
- Provides clear explanations of classification

Key Features:

- Intelligent question classification using LLM
- Automatic routing to specialist agents
- Interactive terminal interface
- Formatted, user-friendly output

3.2 Question Classification System

Type	Category	Description	Routed To
1	Needle in Haystack	Specific facts, dates, numbers, exact values	Specific Task Expert
2	Broad Questions	Summaries, timelines, overviews, trends	Summarization Expert
3	Other	General conversation, greetings, commands	Standard Response

Table 1: Question Classification System

3.3 Summarization Expert Agent

Purpose: Handle broad, complex questions requiring comprehensive analysis

Capabilities:

- Generate general summaries of insurance claims
- Create timeline-based analyses
- Produce executive summaries
- Generate technical overviews
- Analyze trends and patterns

Specialized Prompting: Each summary type uses optimized prompts for better results

3.4 Specific Task Expert Agent

Purpose: Answer precise, fact-based questions

Features:

- Hierarchical auto-merging retrieval
- Metadata-first answering
- Low temperature setting (0) for consistent results
- MCP tool integration for date/time calculations
- Business days calculation
- Policy compliance checking

3.5 ChromaDB Vector Database

Collections:

1. **small_chunks**: Hierarchical 128-character leaf nodes
 - Fine-grained document segmentation
 - Optimal for precise question answering
2. **insurance_claims**: Smart LLM-based chunks
 - Semantically aware chunking
 - Section-aware segmentation
 - Ideal for broad questions and summaries

Advantages:

- Semantic similarity search
- Fast retrieval of relevant information
- Scalable document storage

4. Data Processing Pipeline

1. **PDF Ingestion**: Process insurance claim PDF documents
2. **Smart Chunking**: Intelligently divide content into semantic sections
3. **Vector Embedding**: Convert chunks to embeddings using OpenAI
4. **ChromaDB Storage**: Store embeddings in vector collections
5. **Hierarchical Indexing**: Build 3-level hierarchical index with auto-merging
6. **Query Processing**: User question routed through orchestrator
7. **Retrieval & Generation**: Retrieve relevant chunks and generate response
8. **Response Formatting**: Present answer to user

5. Workflow Example

User Question: "Summarize this insurance claim"

1. **Classification**: Orchestrator identifies as Type 2 (Broad Question)
2. **Routing**: Question forwarded to Summarization Expert Agent
3. **Retrieval**: Summarization Expert queries ChromaDB for relevant chunks
4. **Generation**: LLM generates comprehensive summary
5. **Response**: Formatted summary returned to user

6. Key Technologies

- **LangChain**: Multi-agent orchestration framework
- **ChromaDB**: Vector database for semantic search
- **LlamaIndex**: Hierarchical document indexing and retrieval
- **OpenAI API**: GPT-3.5-turbo for classification and generation
- **MCP (Model Context Protocol)**: Tool integration for compliance calculations
- **Python 3.8+**: Primary programming language

7. Configuration & Environment

Required Environment Variables:

- `OPENAI_API_KEY`: Your OpenAI API key
- `MODEL_NAME`: LLM model (default: gpt-3.5-turbo)
- `TEMPERATURE`: Response creativity (default: 0.7)

Installation Steps:

1. Install required packages: `pip install -r requirements.txt`
2. Configure environment variables in `.env` file
3. Process PDF: `python chromadb_chunk_pdf.py`
4. Build hierarchical index: `python hierarchical_retriever.py`
5. Run orchestrator: `python orchestrator_agent.py`

8. Use Cases

- **Claim Summarization**: Get quick overviews of insurance claims
- **Timeline Analysis**: Understand chronological sequence of events
- **Data Extraction**: Retrieve specific claim details and numbers
- **Compliance Checking**: Verify claim meets regulatory requirements
- **Documentation Support**: Answer questions about claim process
- **Trend Analysis**: Identify patterns across multiple claims

9. Security Considerations

- Never commit `.env` file or expose API keys
- Use `.env.example` as a template only
- Add `.env` to `.gitignore`
- Revoke any exposed API keys immediately
- Secure storage of PDF documents
- API key encryption in production environments

10. Future Enhancements

1. **Advanced NLP**: Multi-language support for international claims
2. **Machine Learning**: Predictive analytics for claim processing
3. **Real-time Monitoring**: Live dashboard for claim metrics
4. **Integration**: Connect with external insurance systems
5. **Scalability**: Support for large-scale claim processing
6. **Advanced Tools**: Enhanced MCP tools for specialized calculations

Conclusion

The Insurance Claim RAG System represents a sophisticated approach to intelligent document processing and question-answering. By combining multi-agent orchestration, intelligent chunking strategies, and hierarchical retrieval, the system can handle both precise fact-based questions and broad analytical queries efficiently. The modular architecture allows for easy extension and customization based on specific insurance domain requirements.

System Version: 1.0
Last Updated: January 2026
Documentation: Complete System Architecture